SPRINT 03 · ANGULAR · LAB01

Angular Basics

TypeScript, CLI, componentes, comunicación, servicios, pipes, directivas y ciclo de vida

Ejercicios de entrenamiento. Hay tres niveles y cada uno es un proyecto distinto e independiente, así que no necesitas el anterior para empezar. Los enunciados no incluyen la solución: usa «Cómo saber que está bien» para comprobar tu trabajo.

Nivel	Proyecto	Practicas	Tiempo orientativo
Fácil	Agenda de contactos	TypeScript, Angular CLI, interpolación, `@if`, `@for` y two-way binding	45-60 min
Medio	Biblioteca personal	Comunicación entre componentes, pipes propios, inmutabilidad y `ngOnChanges`	1,5-2 h
Difícil	FitTrack	Inyección de dependencias, comunicación entre hermanos, directivas de atributo y ciclo de vida	3-4 h

> Trabaja con Angular 22, componentes standalone, `@if` / `@for` e `inject()`. Crea cada proyecto por separado con `ng new <nombre> --style=scss --ssr=false`. Si prefieres decoradores (`@Input`, `@Output`) o funciones (`input()`, `output()`), ambas opciones valen. Angular 22 no usa Zone.js: guarda en un `signal` el estado que cambie fuera de un evento de plantilla (temporizadores, HTTP o suscripciones al router), o la vista no se actualizará.

Fácil

45-60 min

`agenda-contactos`

Agenda de contactos

Practicas: TypeScript, Angular CLI, interpolación, `@if`, `@for` y two-way binding

Vas a crear una agenda sencilla que muestra contactos y permite buscarlos por nombre. Los datos están en memoria.

Modelo

```
export interface Contacto {
  id: number;
  nombre: string;
  email: string;
  telefono?: string;
  favorito: boolean;
}
```

Requisitos

1. Crea el proyecto y guarda la interfaz en `src/app/models/contacto.ts`.
2. Genera con la CLI el componente `contacto-lista` y úsalo en `App`.
3. Define un array con al menos cinco contactos. Deja dos sin teléfono y marca dos como favoritos.
4. Escribe dos funciones tipadas: `favoritos(lista: Contacto[]): Contacto[]` y `buscarPorNombre(lista: Contacto[], texto: string): Contacto[]`. La búsqueda no distingue mayúsculas.
5. Pinta la lista con `@for` y un `track` adecuado. Muestra el teléfono solo si existe y añade un mensaje con `@empty` cuando no haya resultados.
6. Añade un buscador con `[(ngModel)]` enlazado en dos direcciones y un contador «X de Y contactos».

Detecta los errores de tipos

Sin ejecutar nada, indica qué constantes no compilan y por qué:

```
const a: Contacto = { id: '1', nombre: 'Ana', email: 'ana@mail.com', favorito: true };
const b: Contacto = { id: 2, nombre: 'Luis', favorito: false };
const c: Contacto = { id: 3, nombre: 'Eva', email: 'eva@mail.com', telefono: 600111222, favorito: false };
const d: Contacto = { id: 4, nombre: 'Iker', email: 'iker@mail.com', favorito: false };
```

Cómo saber que está bien

- Escribir «AN» encuentra a «Ana» y el contador se actualiza al momento.
- Una búsqueda sin coincidencias muestra el mensaje de lista vacía.
- `ng build` termina sin errores de compilación.

Preguntas

- ¿Qué falla si no importas `FormsModule`? ¿Por qué ese error aparece solo en este componente?
- ¿Para qué sirve `track` y qué cambia al usar `contacto.id` frente a `$index`?

Medio

1,5-2 h

`biblioteca-personal`

Biblioteca personal

Practicas: Comunicación entre componentes, pipes propios, inmutabilidad y `ngOnChanges`

Una biblioteca personal registra los libros que lees y el progreso de cada uno. Un componente padre guarda los datos y varias tarjetas hijas los muestran.

Modelo

```
export type EstadoLibro = 'pendiente' | 'leyendo' | 'terminado';

export interface Libro {
  id: number;
  titulo: string;
  autor: string;
  paginas: number;
  paginasLeidas: number;
  estado: EstadoLibro;
}
```

Requisitos

1. Crea `LibroCard`, que recibe un `libro` de entrada y muestra título, autor, estado y progreso.
2. Crea el pipe `progreso`: `{{ libro | progreso }}` devuelve texto como `45 %`, sin decimales. Si `paginas` es 0, devuelve `0 %`.
3. Añade a la tarjeta los botones «+10 páginas» y «Marcar como terminado». El hijo no modifica el libro: emite un evento y el padre decide.
4. El padre actualiza su lista **sin mutarla** (crea un array y un objeto nuevos). `paginasLeidas` nunca supera a `paginas`. Al llegar al final, el estado pasa a `terminado`.
5. Añade filtros por estado: «Todos», «Pendientes», «Leyendo» y «Terminados».
6. Muestra un resumen con los libros terminados y el total de páginas leídas.
7. En `LibroCard`, implementa `ngOnChanges` y registra en consola el progreso anterior y el nuevo.

Cómo saber que está bien

- Pulsar «+10 páginas» en el último tramo deja el libro en el 100 % y como terminado.
- Los filtros y el resumen siguen siendo correctos después de cada acción.
- La consola muestra cada cambio, con el valor anterior y el nuevo.

Preguntas

- Cambia temporalmente el padre para mutar el objeto (`libro.paginasLeidas += 10`). ¿Se ejecuta `ngOnChanges`? ¿Por qué?
- ¿Por qué tu pipe puede ser puro? ¿Qué tendrías que cambiar si recibiera un objeto mutado?

Difícil

3-4 h

`fittrack`

FitTrack

Practicas: Inyección de dependencias, comunicación entre hermanos, directivas de atributo y ciclo de vida

FitTrack es un panel de entrenamiento con tres componentes hermanos: formulario, historial y resumen semanal. No se pasan datos a través del padre: todos comparten un servicio.

Modelo

```
export type Intensidad = 'baja' | 'media' | 'alta';

export interface Sesion {
  id: number;
  actividad: string;
  minutos: number;
  intensidad: Intensidad;
  fecha: string; // formato AAAA-MM-DD
}
```

Requisitos

1. Crea `SesionesService` (`providedIn: 'root'`) con un `BehaviorSubject<Sesion[]>` privado, un `sesiones$` público de tipo Observable y los métodos `anadir()` y `eliminar()`.
2. Crea los hermanos `SesionForm`, `SesionHistorial` y `ResumenSemanal`. El padre solo los coloca en la plantilla; no les pasa datos.
3. `SesionHistorial` y `ResumenSemanal` se suscriben en `ngOnInit` y cancelan la suscripción en `ngOnDestroy`.
4. Crea el pipe `duracion`: `95` se convierte en `1 h 35 min`, `45` en `45 min` y `60` en `1 h`.
5. Crea la directiva `appIntensidad`, que recibe la intensidad y colorea el borde del elemento. Debe reaccionar si el valor cambia.
6. Añade un botón «Ocultar resumen» que destruya y vuelva a crear `ResumenSemanal` con `@if`. Comprueba en consola que se llama a `ngOnDestroy` y que, al volver, el resumen muestra los datos actuales.
7. Crea `TemporizadorDescanso`: una cuenta atrás de 60 segundos con `setInterval` que se limpia en `ngOnDestroy`. Guarda los segundos en un `signal`; la nota de abajo explica por qué.

Nota sobre Angular 22

> Las aplicaciones nuevas no usan Zone.js: Angular no sabe que un `setInterval` ha cambiado un campo normal y la vista no se refresca. Con `signal()` se entera del cambio. Lo verás a fondo en el LAB04.

Cómo saber que está bien

- Añadir una sesión actualiza el historial y el resumen sin pasar por el padre.
- Tras ocultar y mostrar el resumen, los datos son correctos y solo hay una suscripción activa.
- Si destruyes el temporizador antes de acabar, la cuenta atrás se detiene.

Preguntas

- ¿Qué ocurriría con un `Subject` normal al volver a crear `ResumenSemanal`? ¿Por qué `BehaviorSubject` lo resuelve?
- ¿Por qué la directiva debe aplicar el color en `ngOnChanges` y no en el constructor?
- ¿Qué pasa si eliminas el `unsubscribe` y repites varias veces ocultar y mostrar? ¿Cómo lo demostrarías?